

Práctica 4. Procesamiento audio

Procesamiento de Imágenes, Audio y Vídeo

Luna Yue Hernández Guerra, Jorge Lorenzo Lorenzo

Noviembre 2025

GitHub

Resumen

En esta práctica se profundiza en el procesado de señales de audio mediante la identificación de notas musicales, distinguiendo entre frecuencias fundamentales y armónicos, y la aplicación de filtros a través de una interfaz gráfica, observando los cambios de las señales mediante sus representaciones en el dominio temporal y el dominio de la frecuencia.

1. Enunciados

1.1. Construir un identificador de notas musicales.

Para el identificador de notas musicales, creamos dos clases auxiliares (*AudioProcessor*, *NoteConverter*) y una principal (*SingleNoteIdentifier*).

AudioProcessor se encargará de procesar archivos de audio y analizar su contenido en el dominio de las frecuencias.

Por otro lado, *NoteConverter* es la encargada de convertir una frecuencia (en Hz) en una nota musical reconocible con su nombre en español.

Finalmente, *SingleNoteIdentifier* permite identificar la nota musical que se escucha en un archivo de audio. Para ello, utiliza las funcionalidades de las clases definidas anteriormente, *AudioProcessor* y *NoteConverter*.

La identificación de la nota se realiza de la siguiente forma: buscando la frecuencia fundamental. Se analiza el audio para identificar las frecuencias principales del sonido. Para ello, usa la **Transformada Rápida de Fourier** (FFT), que descompone el sonido en sus diferentes frecuencias. Después, busca los picos más altos en el espectro (las frecuencias más fuertes) y los analiza para quedarse con la frecuencia base, descartando los armónicos.

1.2. Construir una pequeña aplicación que permita operar con diferentes filtros y trabajar con varios umbrales.

En este ejercicio desarrollamos una aplicación interactiva utilizando *Streamlit* que permite aplicar distintos tipos de filtros y umbrales a señales de audio. Asimismo, en la aplicación se muestra tanto la señal original como la filtrada en el dominio temporal y en el dominio de la frecuencia. Estas visualizaciones permiten analizar de forma simultánea los efectos del filtrado sobre la forma de onda y sobre el espectro de frecuencias.

La aplicación incluye un panel lateral desde el cual se pueden seleccionar la fuente de la señal (nota sintética, con frecuencia fundamental y número de armónicos seleccionables; audio WAV; o micrófono del sistema), el tipo de filtro (pasa-bajo, pasa-alto, pasa-banda o rechaza-banda), su implementación (*Butterworth*, *Chebyshev I* y *II* o *Elíptico*), y los parámetros característicos de cada uno, como el orden, las frecuencias de corte y las bandas de paso o parada. Además, incluye la opción de añadir ruido blanco con un nivel de relación señal-ruido (SNR) ajustable, lo que permite analizar la eficacia de los filtros frente a señales contaminadas por ruido. De este modo, se pueden observar los efectos de distintas configuraciones de manera dinámica y visual.

El procesamiento de las señales se realiza mediante funciones auxiliares implementadas en el módulo `utils.py`. Estas funciones se encargan del diseño de los filtros y del tratamiento de las señales, garantizando que las frecuencias de corte se mantengan dentro de los límites válidos del teorema de *Nyquist*. Se emplean tanto los métodos `lfilter` como `filtfilt` para aplicar los filtros y se normalizan las señales resultantes para evitar saturación.

Finalmente, como aportes adicionales, la aplicación permite la reproducción de audio de las señales antes y después del filtrado directamente desde la interfaz, así como la descarga del resultado procesado en formato WAV. También se ha implementado compatibilidad con la entrada de audio en tiempo real desde el micrófono, lo que posibilita experimentar con la voz u otras fuentes sonoras.

2. Dificultades

Durante el desarrollo del identificador de notas musicales encontramos varias dificultades.

En primer lugar, uno de los principales retos fue encontrar un conjunto de datos de notas reales válido para el ejercicio. Inicialmente habíamos escogido una página web que tenía archivos en formato `.aiff` que había que convertir a `.wav` mediante herramientas externas, lo que incrementaba el tiempo de preprocesamiento. Finalmente, encontramos un nuevo *dataset* con datos listos para procesar, en formato `.wav` de un solo canal y sin silencios ni ruidos, lo que facilitó su uso.

Otra dificultad importante fue la detección de la frecuencia fundamental, ya que en muchas grabaciones los armónicos aparecían con mayor intensidad que la propia frecuencia base. Esto provocaba que el sistema confundiera notas o asignara octavas incorrectas.

Por otro lado, en el proceso de creación de una aplicación para operar con diferentes filtros, también se encontraron ciertas dificultades. Primero, decidir cómo y dónde íbamos a presentar la aplicación. Tras decantarnos por utilizar la herramienta *Streamlit*, la siguiente dificultad fue introducir la opción de grabar el audio con micrófono. Pues en un principio teníamos ciertos errores con algunas librerías y tuvimos que seleccionar otra.

3. Conclusión

En conclusión, este trabajo permitió comprender y aplicar conceptos fundamentales del procesamiento de señales, desde la identificación automática de notas musicales mediante análisis espectral hasta el diseño y la aplicación práctica de filtros de audio.